

EMENTA PEDAGÓGICA - DATA SCIENCE + IA

1 - Dados, IA e Negócio: o diferencial do cientista de dados moderno

Professor responsável: **Renata Biaggi**

Titulação: **Pós-graduação**

O papel do cientista hoje

- Apresentação do corpo docente e da pós-graduação
- Os diferentes perfis de entrada e como cada um aproveita melhor a formação
- Carreiras em dados – o que cada um faz e qual perfil essa pós está formando
- O mercado de dados hoje e tendências para o futuro: o que as empresas contratam, salários, setores e trajetórias reais no Brasil e no mundo
- Machine Learning x IA
- Como a IA generativa mudou o trabalho do cientista de dados
- Como o curso funciona: estrutura, cronograma, o porquê de cada disciplina e o que o aluno vai sair sabendo
- O projeto capstone e os projetos de aula: como o portfólio é construído ao longo da pós
- A importância do portfólio

Business Analytics

- Interface entre análise x ciência x negócios
- Como funciona um projeto em dados
- Introdução ao CRISP-DM adaptado: framework que orienta projetos de ciência de dados
- O papel do cientista de dados como tradutor entre dados e decisão de negócio
- Como transformar um problema de negócio em um problema de dados: o framework de perguntas certas
- Quando dados resolvem o problema e quando não resolvem
- Métrica norte: o que é, como escolher e por que toda empresa deveria ter uma

- Métricas de guardrail: o que não pode piorar enquanto você otimiza a métrica norte
- KPIs operacionais vs KPIs de modelo vs KPIs estratégicos: a diferença prática
- Armadilhas clássicas de métricas: vaidade, proxy ruim e otimização do indicador errado
- Como estimar o impacto financeiro de um projeto de dados antes de construir
- Leitura de casos reais: como empresas resolveram problemas de negócio com dados
- Decisão automatizada vs decisão assistida por IA: quando usar cada abordagem e as implicações para o negócio

Entendendo os dados com estatística

- Tipos de variáveis e escalas de medição: nominal, ordinal, intervalar e razão
- Medidas de tendência central: média, mediana e moda — quando cada uma importa
- Medidas de dispersão: variância, desvio padrão, IQR e coeficiente de variação
- Distribuições de frequência, histogramas, boxplots e violin plots
- Correlação: Pearson, Spearman e cuidados na interpretação
- Correlação vs causalidade: o erro mais caro em projetos de dados
- Detecção e tratamento de outliers: Z-score, IQR e decisões justificadas
- Análises de negócios: RFM, Cohort e funil
- Limpeza de dados: o que precisamos tratar antes de analisar
- Prática aplicada com ferramenta de IA para código - análise completa com o olhar de um cientista de dados

GitHub como portfólio

- O que é Git e GitHub e por que todo cientista de dados precisa
- Configuração inicial: instalação, git config, SSH key
- O fluxo básico: init, add, commit, push, pull
- .gitignore para projetos de dados: o que nunca versionar
- Estrutura de repositório profissional: pastas, README e convenções
- README como vitrine: como escrever um README que recrutador lê
- Primeiro repositório da pós publicado

2 - SQL para cientistas de dados

Professor responsável: **Salumita Dantas**

Titulação: **Pós-Graduação**

Conteúdo programático:

Fundamentos de bancos de dados relacionais

- O que são bancos de dados relacionais e por que dados vivem em tabelas
- SGBDs e seus contextos de uso: MySQL, PostgreSQL e BigQuery
- Modelagem conceitual, lógica e física: entidades, atributos e relacionamentos
- Normalização e formas normais
- Modelagem dimensional: fato, dimensão, star schema e snowflake
- Data Warehouse vs Data Lake

SQL para análise de dados

- Comandos de seleção: SELECT, FROM, WHERE, DISTINCT e LIMIT
- Operadores aritméticos, de comparação e lógicos
- Junções: JOIN, LEFT JOIN, RIGHT JOIN e FULL JOIN
- Funções de agregação, GROUP BY e HAVING
- Subqueries e CTEs: queries modulares, reutilizáveis e legíveis
- Funções de data e texto: manipulando timestamps e strings em dados de transação

Window functions — o diferencial analítico do SQL

- O que são window functions e por que transformam a capacidade analítica do DS
- PARTITION BY e ORDER BY: definindo a janela de cálculo
- Funções de ranking: ROW_NUMBER, RANK e DENSE_RANK — diferenças e quando usar cada uma
- Funções de deslocamento: LAG e LEAD — comparando valores entre períodos e identificando sequências
- Funções de agregação sobre janelas: SUM, AVG, MIN e MAX sem colapsar linhas
- Frames de janela: ROWS e RANGE — controlando o intervalo da janela

- Aplicações analíticas reais: médias móveis, totais acumulados, percentuais sobre total e comparativos período a período

SQL analítico avançado

- Views: quando encapsular queries recorrentes e boas práticas de nomenclatura
- Materialized Views: armazenando resultados de queries pesadas — contexto e quando fazem sentido
- Índices
- Planos de execução, como ler o EXPLAIN, identificar gargalos e tomar decisões baseadas em evidência
- Boas práticas analíticas: queries legíveis, performáticas e sustentáveis em produção

Capstone — Fraude

- Exploração do dataset de transações financeiras via SQL
- Identificando padrões suspeitos com window functions: sequências de transações, variações de comportamento e anomalias temporais
- Construção de KPIs de fraude: taxa de fraude, ticket médio e volume por período
- Views analíticas para consumo recorrente das métricas de fraude

3 - Python para cientistas de dados

Professor responsável: **Cláudio Raposo**

Titulação: **Mestrado**

Noções de computação e IA para código

- O que é Python e por que é a linguagem do cientista de dados
- Por que Python depois de SQL - diferenças práticas
- O que SQL não consegue responder e onde Python entra
- Como os computadores funcionam e o que é linguagem de programação nesse contexto
- Sistemas operacionais para ciência de dados: Windows, Linux e macOS — diferenças práticas

- Terminal e linha de comando: navegação, manipulação de arquivos e automação básica
- Apresentando o VSCode: extensões, atalhos e configuração produtiva
- Google Colab: quando usar e limitações em projetos reais

Aprofundamentos

- Variáveis e tipos de dados: int, float, string, bool
- Operadores aritméticos, lógicos e de comparação
- Condicionais: if, elif, else
- Loops: for e while — quando usar cada um
- Funções: definindo, chamando, parâmetros e retorno
- Escopo de variáveis: local vs global
- Tratamento de erros: try, except e raise
- Boas práticas: nomes claros, comentários e PEP8
- Como usar IA para escrever, entender e depurar código sem perder o raciocínio próprio

Estruturas de dados

- Listas: criação, indexação, fatiamento, métodos essenciais
- Tuplas: quando usar em vez de listas
- Dicionários: chave-valor, acesso, iteração e métodos
- Sets: operações de conjunto e casos de uso
- Compreensão de listas e dicionários: sintaxe e quando aplicar

Manipulação de arquivos e dados externos

- Lendo e escrevendo arquivos: txt, CSV e JSON
- Bibliotecas padrão úteis: os, sys, datetime e re
- Introdução a APIs REST: fazendo requisições com requests

NumPy

- Arrays vs listas: por que arrays são mais eficientes para dados
- Indexação, fatiamento e arrays multidimensionais
- Operações vetorizadas: soma, multiplicação e funções matemáticas

Funções estatísticas: mean, std, min, max e percentile

Pandas

- Series e DataFrames: criação e estrutura
- Carregando dados: CSV, Excel e JSON
- Seleção e filtragem: loc, iloc e condições booleanas
- Ordenação e renomeação de colunas
- Limpeza de dados: valores ausentes, duplicatas e tipos inconsistentes
- Transformações: apply, map e funções lambda
- Agrupamentos: groupby e funções de agregação
- Combinando dados: merge e concat
- Reestruturando: pivot, melt e stack

Visualização de dados

- Matplotlib: gráficos estáticos, subplots e customização
- Seaborn: visualizações estatísticas com poucas linhas
- Plotly: gráficos interativos para exploração
- Escolha do gráfico certo: bar, line, scatter, heatmap, boxplot e violin
- Boas práticas: títulos, eixos, cores e legibilidade

Gerenciamento de pacotes e ambientes virtuais: pip, conda, virtualenv e UV/poetry

Capstone — Fraude

- Carregamento, limpeza, análise univariadas e bivariadas
- Visualizações
- Atualização do repositório no GitHub

4 - Decisão com Dados: Estatística, Inferência e Testes A/B

Professor responsável: **Renata Biaggi**

Titulação: **Pós-graduação**

Conteúdo programático:

Probabilidade e distribuições

- Noções de probabilidade: eventos, espaço amostral e regras básicas
- Probabilidade condicional e Teorema de Bayes
- Variáveis aleatórias discretas: distribuição uniforme, binomial, Poisson, geométrica, binomial negativa e hipergeométrica
- Variáveis aleatórias contínuas: distribuição normal, exponencial, t de Student e qui-quadrado
- Função de densidade, CDF e como usar distribuições para responder perguntas de negócio
- Lei dos grandes números e Teorema Central do Limite

Inferência estatística

- Amostra vs população: conceitos de amostragem e representatividade
- O que é um intervalo de confiança
- O que é, como calcular e o que afeta a margem de erro
- Intervalos de confiança para médias e proporções
- O que afeta a largura do IC
- Trade-off entre precisão e confiança: por que 99% de confiança não é sempre melhor
- Conexão com testes de hipótese
- Prática: calculando e interpretando intervalos de confiança

Conceitos importantes para testes de hipótese

- Erros tipo I e tipo II
- P-valor, nível de significância e tamanho de efeito
- Poder estatístico e cálculo de tamanho mínimo de amostra

Testes de hipótese paramétricos

- Testes de hipótese paramétricos - média, proporção e variância, 1 ou 2 amostras
- ANOVA 1 fator: comparação de múltiplos grupos
- Bonferroni e correção para múltiplas comparações
- Prática: calculando e interpretando o p-valor e realizando múltiplas comparações

Testes de hipótese não paramétricos

- Como funcionam os testes não paramétricos
- Quando usar testes não paramétricos: critérios e trade-offs
- Exemplos aplicados e o correspondente paramétrico
- Bootstrap: resampling para estimação robusta de intervalos de confiança
- Prática: aplicação de testes não paramétricos

Experimentação e Teste A/B

- O que é um experimento controlado e por que é o padrão ouro para causalidade em dados
- Diferença entre correlação observacional e efeito causal: por que precisamos de experimentos
- Unidade de randomização: usuário, sessão, conta ou transação — a escolha e suas implicações
- Grupo de controle e grupo de tratamento: como garantir comparabilidade entre os grupos
- Métricas de guardrail e métricas de sucesso: o que monitorar além da métrica principal
- Critérios de parada: quando encerrar o experimento — por tempo ou por amostra atingida

Cálculo de tamanho de amostra

- Efeito mínimo detectável (MDE): a menor diferença que faz sentido detectar para o negócio
- Poder estatístico
- Fórmula do tamanho de amostra: como cada parâmetro afeta o resultado
- Calculando tamanho de amostra para médias e para proporções
- Impacto da variabilidade dos dados no tamanho necessário: quanto mais ruidosos os dados, mais amostra precisamos
- Prática: desenhando um teste AB

Armadilhas clássicas em experimentação

- Data peeking, Multiple testing problem
- Correção de Bonferroni e FDR

- Novelty effect
- Viés de seleção na randomização
- Simpson's Paradox: quando o resultado agregado contradiz o resultado por subgrupo
- Contaminação entre grupos

Capstone — Fraude

- Análise completa no projeto de fraude, com testes de hipótese
- Atualização do repositório no GitHub

5 - Fundamentos de Machine Learning Aplicado a Problemas Reais

Professor responsável: **Renata Biaggi**

Titulação: Pós-graduação

Conteúdo programático:

O que é Machine Learning e como ele se encaixa em projetos reais

- Diferença entre Inteligência Artificial, Machine Learning e Deep Learning: onde cada um começa e termina
- Projetos de IA vs projetos de Analytics: quando usar cada abordagem e como escolher
- O papel do cientista de dados no ciclo de vida de um projeto de ML
- Tipos de aprendizado: supervisionado, não supervisionado e por reforço — visão geral
- Erros clássicos de quem começa em ML: resolver o problema errado com o modelo certo

CRISP-DM e mentalidade de projeto

- As seis fases do CRISP-DM: entendimento do negócio, dos dados, preparação, modelagem, avaliação e deploy
- Como transformar uma dor de negócio em um problema de ML bem definido
- Definição do target: a decisão mais importante de qualquer projeto de ML

- Metas de negócio vs metas analíticas: como alinhar os dois mundos
- Mapa mental de Machine Learning: algoritmos, métricas e decisões organizadas

Preparação de dados para Machine Learning

- Por que a preparação de dados é a etapa mais importante e mais negligenciada
- Tratamento de valores ausentes: estratégias, implicações e decisões justificadas
- Tratamento de outliers: detecção, contexto de negócio e quando não tratar
- Data leakage: o que é, como acontece silenciosamente e como evitar
- Scaling de variáveis numéricas: normalização (MinMax), padronização (StandardScaler) e RobustScaler — quando usar cada um
- Encoding de variáveis categóricas: one-hot encoding, label encoding, ordinal encoding
- Técnicas avançadas de encoding: target encoding, weighted target encoding e K-fold target encoding — como evitar leakage no processo
- Intuição sobre o pré-tratamento necessário para cada tipo de algoritmo
- Aplicações práticas de cada tratamento

Conceitos críticos que definem a qualidade de um modelo

- Overfitting e underfitting: diagnóstico visual e interpretação das curvas de aprendizado
- Split out-of-time: por que o split temporal é obrigatório em dados com dependência cronológica
- Como o desbalanceamento afeta métricas e decisões de modelo
- Dados desbalanceados: SMOTE, undersampling, oversampling e class weights — trade-offs de cada abordagem
- Aplicações práticas e comparações entre critérios

Técnicas de avaliação de modelo - Split de dados

- Por que o erro de treino mente: a necessidade de dados não vistos
- Hold-out validation: limitações e quando ainda faz sentido
- K-Fold Cross-Validation: como funciona, variância das estimativas e custo computacional
- Stratified K-Fold: preservando proporções de classes em dados desbalanceados

- Out-of-time split: validação temporal obrigatória em séries com ordem cronológica
- Leave-One-Out (LOO): quando usar e quando evitar
- Aplicações práticas de técnica de validação

Capstone — Fraude

- Pipeline de pré-processamento completo: da ingestão bruta aos datasets prontos para ML
- Atualização do repositório no GitHub

6 - Modelos Supervisionados: Da Previsão à Decisão

Professor responsável: **Vanessa Gaigher**

Titulação: **Mestrado**

Conteúdo programático:

Regressão Linear

- Intuição geométrica: o que o modelo realmente faz no espaço de features
- Derivadas e gradiente descendente: como o modelo aprende iterativamente
- Função de custo MSE: o que minimizamos e por quê
- Coeficientes e interpretação: o que cada peso significa em termos de negócios
- Métricas de avaliação: MAE, RMSE, R^2 e R^2 ajustado — interpretação prática
- Prática: aplicação em case de regressão aplicada com interpretação de coeficientes

Regressão Logística

- Por que regressão linear não funciona para classificação: o problema da fronteira
- A função sigmoid: transformando saída linear em probabilidade
- Log-loss (cross-entropy): a função de custo da regressão logística e sua intuição
- O que a derivada da sigmoid nos diz sobre o aprendizado

- Fronteira de decisão: visualização e interpretação geométrica
- Odds ratio e log-odds: interpretando coeficientes em termos probabilísticos
- Métricas de classificação: accuracy, precision, recall, F1-score, AUC-ROC, curva KS, curva precision-recall
- Prática: regressão logística no case de fraude com análise de métricas

Regularização

- O problema do overfitting em modelos lineares: quando o modelo decora os dados
- Regularizações Ridge (L2), Lasso (L1) e Elastic Net: como funciona cada técnica
- Como escolher o parâmetro de regularização (alpha): validação cruzada na prática
- Prática: comparando modelos com e sem regularização

K-Nearest Neighbors (KNN)

- Intuição: classificando pelo vizinho mais próximo
- Matemática de distâncias: euclidiana, Manhattan, Minkowski e cosseno
- O espaço métrico e por que a escala dos dados importa tanto no KNN
- Escolha do K: trade-off entre viés e variância
- Maldição da dimensionalidade: por que o KNN sofre em alta dimensão
- KNN para regressão vs classificação: diferenças e casos de uso
- Quando KNN faz sentido e quando não faz — limitações computacionais
- Prática: KNN no case de fraude com análise de impacto do K

Naive Bayes

- Teorema de Bayes revisitado e como funciona o Naive Bayes
- Variantes: Gaussian Naive Bayes, Multinomial e Bernoulli — quando usar cada um
- Prática: Naive Bayes no case de fraude

Support Vector Machine (SVM)

- Intuição geométrica: encontrando o hiperplano de maior margem
- Vetores de suporte: quem realmente define a fronteira de decisão
- Hard margin vs soft margin
- Produto interno e o kernel trick: projetando dados em espaços de maior dimensão sem custo computacional

- Kernels principais: linear, polinomial, RBF (Gaussian) e sigmoid
- Parâmetros C e gamma: intuição e impacto no overfitting
- SVM para regressão: SVR e epsilon-insensitive loss
- Limitações
- Prática: SVM no case de fraude com análise de kernels

Capstone — Fraude

- Aplicação do modelos referentes à classificação
- Atualização do repositório no GitHub

7 - Modelos Avançados, Ensembles e Explicabilidade

Professor responsável: **Vanessa Gaigher**

Titulação: **Mestrado**

Conteúdo programático:

MLflow para Rastreamento de Experimentos

- Por que rastreamento de experimentos é essencial em projetos reais
- Conceitos: experimento, run, parâmetros, métricas e artefatos
- Logando modelos, métricas e visualizações automaticamente
- Model Registry: versionamento e promoção de modelos
- Comparando runs e reproduzindo experimentos
- Prática: rastreando os experimentos com MLflow

Árvores de Decisão

- Intuição: particionando o espaço de features com perguntas binárias
- Critérios de split: Gini impurity, entropia e information gain — matemática e intuição
- Profundidade, folhas e poda: controlando complexidade e overfitting
- Interpretabilidade: visualizando e lendo uma árvore de decisão
- Limitações: instabilidade e sensibilidade a pequenas mudanças nos dados

- Prática árvore de decisão no case de regressão

Modelos de Ensemble

- Bagging vs Boosting vs Stacking
- Algoritmos de Ensemble: Random Forest, XGBoost, LightGBM, CatBoost, Stacking
- Prática modelos de ensemble no case de regressão

Tuning de Hiperparâmetros

- O que são hiperparâmetros e por que não podem ser aprendidos pelos dados
- Grid Search: busca exaustiva e seu custo computacional
- Random Search: eficiência com aleatoriedade controlada
- Bayesian Optimization: aprendendo quais regiões do espaço explorar
- Ferramentas automáticas de tuning de hiperparâmetros (ex: optuna)
- Prática tuning de hiperparâmetros no case de regressão

Feature Selection e Feature Importance

- Por que mais features nem sempre é melhor: curse of dimensionality e ruído
- Filter methods: correlação, mutual information e chi-quadrado
- Wrapper methods: RFE (Recursive Feature Elimination) e Sequential Feature Selection
- Embedded methods: Lasso, árvores e importância nativa de modelos
- Permutation importance: avaliando features removendo sua informação
- Feature importance nativa de modelos
- Prática seleção de features no case de regressão

Explicabilidade de Modelos

- Por que explicabilidade importa: regulação, confiança e debugging
- SHAP (SHapley Additive exPlanations): a matemática de teoria dos jogos por trás
- SHAP global: quais features mais impactam o modelo como um todo
- SHAP local: por que o modelo tomou aquela decisão para aquela transação específica
- LIME: aproximações lineares locais para explicar qualquer modelo
- Partial Dependence Plots (PDP) e Individual Conditional Expectation (ICE)
- Prática: explicando previsões no contexto de regressão e explicando para negócios

Classificação Multiclasse

- Extensões da classificação binária: one-vs-rest e one-vs-one
- Métricas multiclasse: macro, micro e weighted averaging
- Matriz de confusão multiclasse: lendo e interpretando
- Desafios: desbalanceamento em múltiplas classes
- Prática: classificação multiclasse

Capstone - Fraude

- Aplicar modelos de ensemble, feature selection e feature importance
- Aplicar técnicas de explicabilidade
- Ajuste de Threshold
- Calibração de Probabilidades

8 - Aprendizado Não Supervisionado, Segmentação e Detecção de Anomalias

Professor responsável: **Vanessa Gaigher**

Titulação: **Mestrado**

Conteúdo programático:

Introdução ao aprendizado não supervisionado

- Diferença fundamental entre supervisionado e não supervisionado: sem rótulo, sem ground truth
- Quando usar aprendizado não supervisionado em problemas reais
- Principais famílias de algoritmos: clusterização, redução de dimensionalidade e detecção de anomalias
- Avaliação sem rótulo: como saber se o resultado é bom quando não há resposta certa

Matemática de distâncias e espaços

- Relembrando distâncias: euclidiana, Manhattan, Minkowski e cosseno
- Autovalores e autovetores: intuição geométrica e o que revelam sobre a estrutura dos dados
- SVD (Singular Value Decomposition): decomposição matricial e suas aplicações em dados

Redução de dimensionalidade

- Por que reduzir dimensões: visualização, performance e remoção de ruído
- Maldição da dimensionalidade: por que distâncias perdem sentido em alta dimensão
- PCA (Principal Component Analysis): decomposição espectral, variância explicada e componentes principais
- Como o PCA usa autovalores e autovetores para encontrar direções de máxima variância
- Escolha do número de componentes: variância acumulada e scree plot
- Kernel PCA: PCA não linear para dados com estrutura complexa
- t-SNE: preservando estrutura local para visualização em 2D e 3D
- UMAP: alternativa ao t-SNE mais rápida, escalável e com melhor preservação global
- Quando usar PCA vs t-SNE vs UMAP: guia prático
- Prática: visualizando o espaço de dados vs legítimas com PCA e UMAP

Clusterização

- O que é clusterização e o que significa um bom cluster
- K-Means: funcionamento interno, inicialização com K-Means++, escolha de K com elbow e silhouette
- K-Medoids: alternativa robusta a outliers — quando faz sentido
- DBSCAN: clusterização por densidade, epsilon e min_samples — detectando clusters de forma irregular
- HDBSCAN: versão hierárquica e robusta do DBSCAN para dados reais
- Clustering hierárquico: agglomerative, divisive, dendrogramas e escolha do número de clusters
- Avaliação de clusters: silhouette score, Davies-Bouldin index e Calinski-Harabasz
- Interpretação de negócio: o que fazer com os clusters depois de encontrá-los
- Prática: segmentando comportamentos

Detecção de anomalias

- Anomalia vs outlier: diferenças conceituais e implicações práticas
- Isolation Forest: isolando pontos raros com partições aleatórias — matemática e intuição
- Local Outlier Factor (LOF): anomalias baseadas em densidade local relativa
- One-Class SVM: modelando a fronteira do comportamento normal
- Combinando detectores: ensemble de anomalias para maior robustez
- Avaliação: precision, recall e AUC quando temos rótulos vs quando não temos
- Prática: detectando anomalias sem usar os rótulos

9 - A Arquitetura da Inteligência Artificial: **Das Redes Neurais aos Modelos Generativos**

Professor responsável: **Salumita Dantas**

Titulação: **Pós Graduação**

Bases e multilayer perceptron

- Dos modelos clássicos às redes neurais
- Motivação Biológica: Anatomia do neurônio e sinapses; o mecanismo de disparo
- Evolução de Arquitetura: Do neurônio único ao MLP e representação hierárquica.
- Fundamentos de Representação: Teorema do aproximador universal.
- Não-Linearidade: Funções de ativação (ReLU, Sigmoid, Tanh).
- Prática Perceptron Multicamadas (MLP) para Classificação

Gradiente e Backpropagation

- Otimização: Gradient descent, taxas de aprendizado e algoritmos de ajuste (Adam, RMSProp).
- Mecânica do Erro: Regra da cadeia e backpropagation passo a passo no grafo de computação.
- Generalização: Dropout, regularização L1/L2 e estratégias contra o overfitting.
- Prática Otimização, Gradiente à Generalização (Classificação) e Mecânica do Erro em Modelos de Regressão

Redes Neurais Convolucionais (CNNs)

- Operações Espaciais: Convolução, filtros (kernels) e parâmetros compartilhados.
- Arquitetura de Visão: Camadas de pooling, stride e padding.
- Evolução SOTA: Estudo de arquiteturas clássicas (AlexNet, VGG, ResNet).
- Laboratório Redes Neurais Convolucionais (CNNs) com VGG16 para Classificação de Imagens
- Transfer learning: reaproveitando modelos pré-treinados sem treinar do zero
- Fine-tuning vs feature extraction: quando retrainar camadas e quando congelar
- Prática Redes Neurais Convolucionais (CNNs) com VGG16 para Classificação de Imagens
- Prática Transfer Learning com InceptionV3 para Classificação de Imagens

Redes para sequências

- Por que dados sequenciais exigem arquitetura diferente
- RNNs: intuição, forward pass recorrente e limitações
- O problema do gradiente desaparecendo em sequências longas
- LSTMs: gates de entrada, esquecimento e saída — como a memória de longo prazo funciona
- GRUs: alternativa mais leve ao LSTM — quando faz sentido
- Prática: Treinamento de uma série temporal com LSTM

Autoencoders

- Arquitetura: encoder, bottleneck e decoder — o que cada parte aprende
- Autoencoders para redução de dimensionalidade: alternativa ao PCA para dados complexos
- Autoencoders para detecção de anomalias
- Denoising Autoencoders: aprendendo representações robustas a ruído
- Variational Autoencoders: introdução à geração probabilística — a ponte para modelos generativos
- Prática Autoencoders (AE) com Fashion MNIST
- Prática Variational Autoencoders com Fashion MNIST

Arquiteturas de Atenção e Transformers

- Da limitação das RNNs à atenção: por que dependências longas quebravam o modelo anterior

- Mecanismo de atenção: queries, keys e values — intuição e matemática
- Scaled dot-product attention e por que o fator de escala importa
- Multi-head attention: múltiplas perspectivas em paralelo
- Positional encoding: injetando ordem sem recorrência
- Arquitetura completa do Transformer: encoder, decoder e as variantes
- Encoder-only (BERT), decoder-only (GPT) e encoder-decoder (T5): diferenças e casos de uso
- Tokenização: BPE e WordPiece — como texto vira input de fato
- Pré-treinamento e fine-tuning: o paradigma que mudou a área
- Prática: usando BERT e GPT-2 para tarefas textuais

Capstone - Fraude

- Aplicar modelos deep learning e contrastar com modelos de machine learning clássico

10 - Inteligência Artificial Generativa - RAGs, Fine-Tuning e estratégias de produção LLMs

Professor responsável: **Danielle Monteiro**

Titulação: **Mestrado**

Conteúdo programático:

Tokenização na prática

- Vocabulários, tokens especiais e janela de contexto: implicações operacionais
- Estimativa e controle de custo via contagem de tokens
- Como a tokenização afeta qualidade de output em tarefas de dados
- Prática: estimando custo e inspecionando tokens em modelos reais

Prompt engineering avançado

- Zero-shot, few-shot e chain-of-thought prompting
- Prompt templates para padronizar fluxos e reutilizar prompts
- Estratégias para reduzir alucinações e melhorar precisão

- Structured outputs: extraindo JSON e dados estruturados de LLMs
- Prompt chaining: encadeando prompts para tarefas complexas

RAG — Retrieval-Augmented Generation

- Arquitetura de RAG: chunking, indexação, retrieval e geração
- Estratégias de chunking: tamanho, overlap e splitting semântico
- RAG avançado: HyDE, RAG fusion e multi-query retrieval
- Avaliação de sistemas RAG: faithfulness, relevância e evals automáticos
- RAG aplicado a dados corporativos: relatórios, documentação e bases internas
- Confidence scoring: estimando a confiabilidade de uma resposta gerada com RAG
- Retrieval fallback: o que fazer quando nenhum documento relevante é encontrado
- Detectando respostas de baixa qualidade: métricas automáticas e heurísticas práticas
- Escalando para humano: quando e como redirecionar para atendimento humano
- Comunicando incerteza ao usuário: como o sistema deve se comportar quando não tem certeza
- Prática: pipeline RAG completo sobre documentos técnicos

Fine-tuning de LLMs

- Quando fine-tuning vale a pena vs RAG vs prompt engineering: o framework de decisão
- Preparação de datasets para fine-tuning: formato JSONL e qualidade dos dados
- LoRA e PEFT: fine-tuning eficiente sem GPU de alta potência
- Avaliação de modelos fine-tunados: métricas e testes A/B
- Fine-tuning via APIs: OpenAI, Anthropic e alternativas
- Prática: fine-tuning com QLoRA em dataset de domínio específico

Guardrails e comportamento seguro em produção

- Tópicos sensíveis: detectando e redirecionando automaticamente assuntos fora do escopo
- Input validation: filtrando perguntas antes de chamar o modelo
- Output validation: verificando qualidade mínima antes de entregar ao usuário

- Circuit breakers: desativando o sistema quando a taxa de respostas ruins ultrapassa um threshold
- Human-in-the-loop: arquiteturas que mantêm supervisão humana em decisões críticas
- Auditoria de falhas: registrando, analisando e aprendendo com os casos em que o sistema errou

Avaliação de Sistemas Generativos

- LLM-as-judge: vieses, calibração e quando confiar
- Métricas por tarefa: sumarização, QA e geração de código
- Detecção de alucinações: abordagens automáticas e seus limites
- Prática: pipeline de avaliação automatizada

11 - Software engineering para Cientistas de Dados

Professor responsável: **Danielle Monteiro**

Titulação: **Mestrado**

Do notebook ao código de produção

- Por que código de notebook não vai para produção: reprodutibilidade, manutenção e colaboração
- O que muda: da experimentação à engenharia
- Estrutura de projeto profissional: organização de pastas, módulos e responsabilidades
- CookieCutter Data Science: template de referência para projetos de dados
- Convenções de nomenclatura e organização que times reais usam

Modularização e boas práticas de código

- Modularização a nível de pastas: separando dados, código, notebooks e outputs
- Modularização a nível de função: funções com responsabilidade única e bem definida

- Princípio DRY (Don't Repeat Yourself)
- Funções puras vs funções com efeitos colaterais: implicações para testabilidade
- PEP8: o guia de estilo do Python e por que seguir convenções importa em times
- Linters: flake8 e ruff para detectar problemas automaticamente
- Formatters: black e isort para padronização automática de código
- Tipagem estática
- Prática de modularização de código com código limpo

Documentação e tratamento de erros

- Docstrings
- Documentando funções de transformação, modelos e pipelines de dados
- Tratamento de exceções: try/except, raise e criando exceções customizadas
- Logging estruturado: níveis, formatação e logging em pipelines de ML
- Diferença entre print e logging: por que logging é obrigatório em produção
- Prática: adicionando documentação e logging no código

Programação Orientada a Objetos para dados

- Classes, atributos e métodos: conceitos fundamentais aplicados a pipelines
- Encapsulamento: protegendo estado interno de transformadores e modelos
- Herança: reutilizando lógica entre diferentes versões de um pipeline
- Polimorfismo: interfaces consistentes para diferentes algoritmos
- Construindo um pipeline orientado a objetos: Transformer, Model e Evaluator como classes
- Integração com scikit-learn: criando transformers customizados compatíveis com Pipeline

Testes para projetos de dados

- Por que testar código de dados: o custo de bugs silenciosos em pipelines
- Testes unitários com pytest: estrutura, fixtures e assertions
- Testando funções de transformação: garantindo que o pré-processamento funciona como esperado
- Testando modelos: validando outputs, shapes e tipos de retorno
- Parametrização de testes: testando múltiplos cenários com uma única função
- Mocks e patches: isolando dependências externas em testes
- Testes de integração: validando o pipeline completo de ponta a ponta

- Testes de dados: validando schema, tipos e distribuições com Great Expectations

Git avançado

- Monorepo vs polirepo: o que são, prós e contras de cada abordagem e como times reais decidem
- Estrutura de repositório para projetos de ML: onde ficam dados, notebooks, código, testes, configs e documentação
- Branches: feature branches, main protegida e convenções de nomenclatura
- Pull requests: como abrir, revisar e dar feedback em código de dados
- Commits semânticos
- Merge vs rebase
- Resolução de conflitos: como acontecem e como resolver sem perder trabalho
- **git stash**: salvando trabalho temporariamente
- **git log** e **git diff**: navegando no histórico do projeto
- Issues e Project Boards: organizando trabalho de dados no GitHub
- Versionando modelos: conectando MLflow com Git para rastreabilidade completa

Integração Contínua

- O que é CI e por que é obrigatório em projetos de ML colaborativos
- GitHub Actions: configurando workflows automáticos
- Rodando testes automaticamente a cada pull request
- Linting e formatação automáticos no pipeline de CI
- Notificações e relatórios de falha

Construção de APIs para modelos de ML

- REST APIs: conceitos, verbos HTTP e design de endpoints para modelos
- FastAPI: construção, validação de input com Pydantic e documentação automática
- Autenticação e autorização
- Rate limiting e controle de acesso em APIs de produção
- Tratamento de erros e respostas padronizadas
- Testes de API com pytest e httpx

Capstone — refatorando o modelo de fraude com OOP

- Transformar o notebook de ML em pacote Python estruturado com classes
- API completa com FastAPI servindo o modelo de fraude

- Entregável: repositório profissional com código testado, documentado e API funcional

12 - MLOps para cientistas de dados: Produtização, Monitoramento e Retreino

Professor responsável: **Danielle Monteiro**

Titulação: **Mestrado**

Conteúdo programático:

Motivação e fundamentos de MLOps

- Por que modelos de ML falham em produção: as causas reais
- A lacuna entre ciência de dados e engenharia: como MLOps fecha esse gap
- Ciclo de vida completo de modelos: do experimento ao retirement
- Níveis de maturidade de MLOps: manual, semi-automatizado e totalmente automatizado
- ML System Design: princípios para sistemas de ML escaláveis e confiáveis
- O custo real de não ter MLOps: retrabalho, inconsistência e modelos degradados

Aprofundamentos em MLflow

- Relembrando: MLflow para Rastreamento de Experimentos
- MLflow Tracking: logando automaticamente modelos, métricas e visualizações
- MLflow Projects: empacotando código para reprodutibilidade
- MLflow Models: formato unificado para servir modelos de qualquer framework
- Relembrando Model Registry: versionamento, staging e promoção de modelos para produção
- Prática: rastreando o pipeline completo do case de fraude com MLflow

Engenharia de features em escala

- O problema de features inconsistentes entre treino e produção
- Feature stores: o que são, por que existem e quando usar
- Feast: arquitetura, conceitos (entity, feature view, data source) e configuração

- Feature point-in-time correctness: evitando leakage temporal em produção
- Alternativas ao Feast
- Prática: implementando feature store para o modelo de fraude

Containers e Docker para MLOps

- O que é Docker e por que é essencial para reprodutibilidade em ciência de dados
- Imagens, containers, volumes e redes: conceitos fundamentais
- Revisão de Docker: imagens, containers, volumes e redes
- Dockerfile para projetos de ML: reprodutibilidade do ambiente de treinamento
- Multi-stage builds: imagens leves para deploy de modelos
- Docker Compose: orquestrando serviços de treino, serving e monitoramento
- Boas práticas: segurança, tamanho de imagem e cache eficiente
- Prática: containerizando o pipeline completo

ML Serving

- Padrões de serving: batch, online e near-real-time — quando usar cada um
- REST vs gRPC: diferenças e casos de uso em serving de modelos
- Latência vs throughput: trade-offs em sistemas de serving
- Model warming e caching: otimizando performance em produção
- A/B testing de modelos em produção: framework e implementação
- Shadow mode: testando novo modelo sem impacto em produção
- Prática: configurando serving completo para o modelo de fraude

CI/CD para Machine Learning

- Diferença entre CI/CD de software e CI/CD de ML: dados, modelos e métricas
- Pipeline de CI para ML: testes de código, testes de dados e testes de modelo
- Pipeline de CD para ML: automatizando treino, validação e deploy
- GitHub Actions para ML: workflows de treino e deploy automáticos
- Triggers de re-treinamento: por tempo, por volume de dados ou por drift detectado
- Prática: pipeline de CI/CD completo

Cloud para MLOps

- Principais provedores: AWS, GCP e Azure — serviços relevantes para ML
- SageMaker (AWS): treinamento, registro e deploy de modelos na nuvem
- Vertex AI (GCP): pipelines de ML gerenciados e Model Registry
- Quando usar cloud vs on-premise: custo, latência e compliance
- FinOps para ML: estimando e controlando custos de treinamento e serving

- Do script local ao projeto cloud-ready: refatorando para rodar em nuvem

Monitoramento de modelos em produção

- Data drift: mudanças na distribuição das features de entrada
- Concept drift: mudanças na relação entre features e target
- Ferramentas de monitoramento
- Alertas: configurando notificações para degradação detectada
- Estratégias de re-treinamento: scheduled, triggered e continuous training
- Champion/challenger: substituindo modelos com segurança em produção
- Prática: implementando monitoramento e governança no modelo de fraude

Capstone — Fraude

- Deploy em cloud com pipeline de CI/CD automatizado
- Monitoramento de drift com alertas configurados
- Estratégia de re-treinamento